Spring Cloud 简介

主要内容

微服务简介
SpringCloud 简介
SpringCloud 框架结构
SpringCloud 和 Dubbo 的对比
SpringCloud 版本号说明

学习目标

知识点	要求
微服务简介	了解
SpringCloud 简介	掌握
SpringCloud 框架结构	掌握
SpringCloud 和 Dubbo 的对比	了解
SpringCloud 版本号说明	掌握

Spring Cloud 简介

一、 微服务简介

1 什么是微服务？

微服务的概念最早是在 2014 年由 Martin Fowler 和 James Lewis 共同提出，他们定义了微服务是由单一应用程序构成的小服务，拥有自己的进程与轻量化处理，服务依业务功能设计，以全自动的方式部署，与其他服务使用 HTTP API 通讯。同时，服务会使用最小规模的集中管理 （例如 Docker）技术，服务可以用不同的编程语言与数据库等。

简单举例：看军事新闻的同学应该都知道，一艘航空母舰作战能力虽然很强，但是弱点太明显，就是防御能力太差，单艘的航空母舰很少单独行动，通常航空母舰战斗群才是主要军事力量，你可以把单艘航母理解为的单体应用（防御差，机动性不好），把航母战斗群（调度复杂，维护费用高）理解为微服务。

2 为什么使用微服务？

2.1 单体应用特点

大部分的开发者经历和开发过单体应用，无论是传统的 Servlet + JSP，还是 SSM，

还是现在的 SpringBoot，它们都是单体应用，那么长期陪伴我们的单体应用有什么弊端？我们是面临了什么问题，导致我们要抛弃单体应用转向微服务架构？主要原因如下：

部署成本高（无论是修改 1 行代码，还是 10 行代码，都要全量替换）。

改动影响大，风险高（不论代码改动多小，成本都相同）。

因为成本高，风险高，所以导致部署频率低（无法快速交付客户需求）。

无法满足快速扩容，弹性伸缩，无法适应云环境特性等问题。

2.2微服务特点

微服务架构的特点：针对特定服务发布，影响小，风险小，成本低；频繁发布版本，快速交付需求；低成本扩容，弹性伸缩，适应云环境。

我们知道一个朴素的理念，没有任何事物是完美的，任何东西都有两面性，有得必有失，那么在选择微服务在解决了快速响应和弹性伸缩的问题同时，它又给我们带来了什么问题？简单总结如下：

分布式系统的复杂性

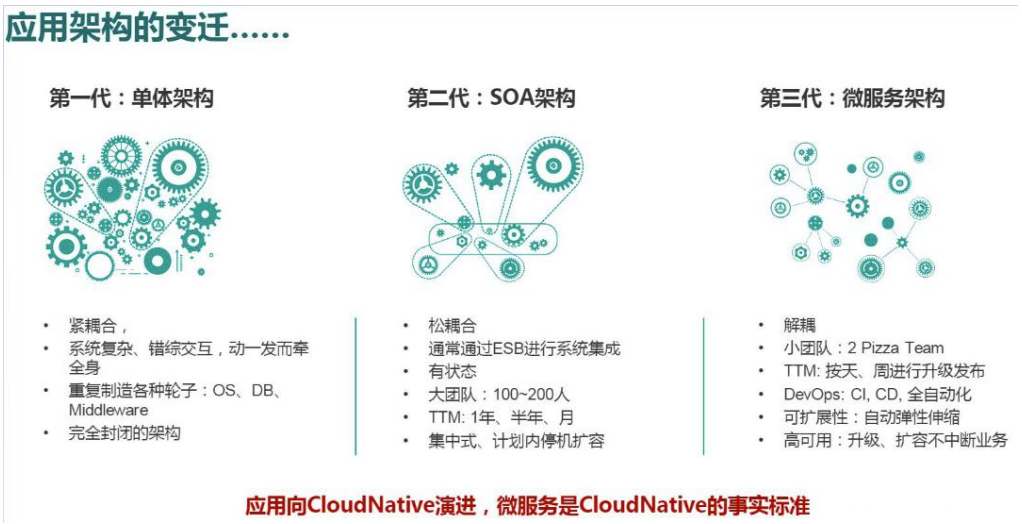
部署，测试和监控的成本问题

分布式事务和 CAP 的相关问题

系统应用由原来的单体变成几十到几百个不同的工程，会所产生例如包括服务间的依赖，服务如何拆封，内部接口规范，数据传递等等问题，尤其是服务拆分，需要团队熟悉业务流程，懂得取舍，要保证拆分的粒度服务既符合“高内聚，低耦合”的基本原则，还要兼顾业务的发展以及公司的愿景，要还要说服团队成员为之努力，并且积极投入，在多方中间取得平衡。

对于分布式系统，部署，测试和监控都需要大量的中间件来支撑，而且中间件本身也要维护，原先单体应用很简单的事务问题 ，转到分布式环境就变得很复杂，分布式事务是采用简单的重试+补偿机制，还是采用二阶段提交协议等强一致性方法来解决，就要取决对业务场景的熟悉加上反复的权衡了，相同问题还包括对 CAP 模型的权衡，总之微服务对团队整体的技术栈水平整体要求更高。

3 应用架构变迁图



二、 Spring Cloud 简介

Spring Cloud 是 Spring 旗下的一个顶级项目。

Spring Cloud 是一个服务治理平台，提供了一些服务框架。包含了：服务注册与发现、配置中心、消息中心 、负载均衡、数据监控等等。

Spring Cloud 是一系列框架的有序集合。它利用 Spring Boot 的开发便利性巧妙地简化了分布式系统基础设施的开发，如服务发现注册、配置中心、消息总线、负载均衡、断路器、数据监控等，都可以用 Spring Boot 的开发风格做到一键启动和部署。Spring Cloud 并没有重复制造轮子，它只是将目前各家公司开发的比较成熟、经得起实际考验的服务框架组合起来，通过 Spring Boot 风格进行再封装屏蔽掉了复杂的配置和实现原理，最终给开发者留出了一套简单易懂、易部署和易维护的分布式系统开发工具包。

Spring Cloud 是一个微服务框架，相比 RPC/SOA 框架而言，Spring Cloud 提供的全套的分布式系统解决方案。

Spring Cloud 对 Netflix 的多个微服务基础框架开源组件进行了封装，同时又实现了和云端平台以及和 Spring Boot 开发框架的集成。

Spring Cloud 为微服务架构开发涉及的配置管理，服务治理，熔断机制，智能路由，微代理，控制总线，一次性 token，全局一致性锁，leader 选举，分布式 session，集群状态管理等操作提供了一种简单的开发方式。

Spring Cloud 为开发者提供了快速构建分布式系统的工具，开发者可以快速的启动服务或构建应用、同时能够快速和云平台资源进行对接。

1 Netflix 简介

Netflix(Nasdaq NFLX) 成立于 1997 年，是一家在线影片租赁提供商，主要提供 Netflix 超大数量的 DVD 并免费递送，总部位于美国加利福尼亚州洛斯盖图。

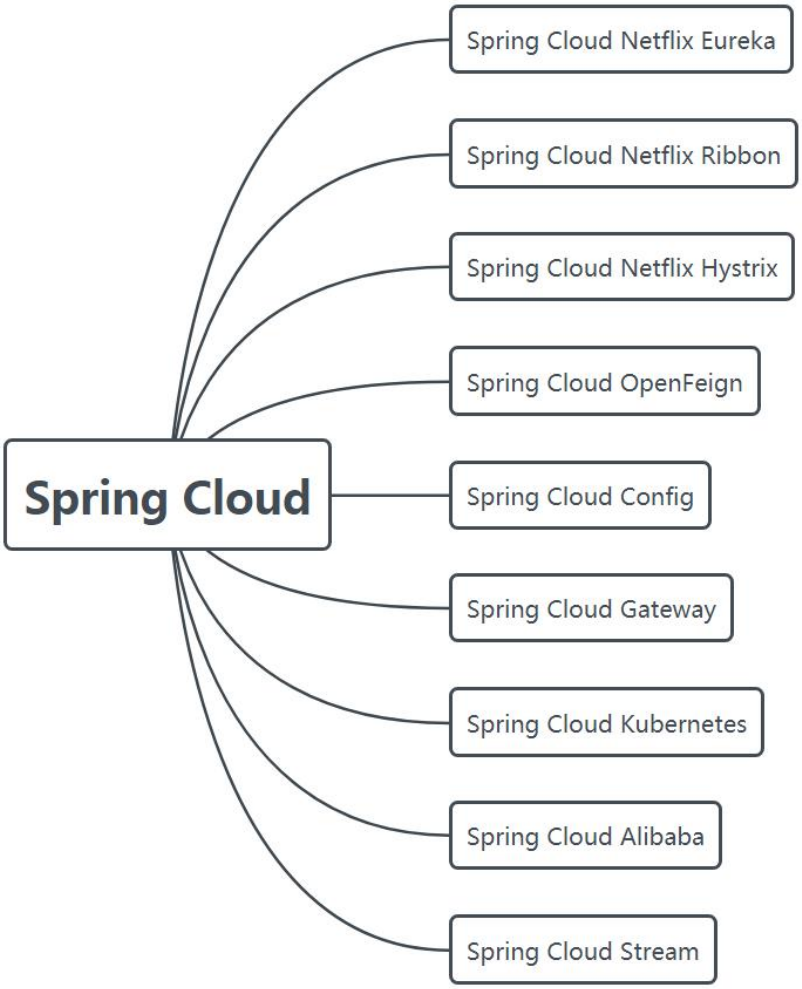
Netflix 经过几年的开源实践，推出最新开源改革计划，打造了全新的 Netflix 开源门户，并且会继续开源更多好项目，增加 Netflix 项目开源透明度。

互联网流媒体播放商 Netflix 在几年前就开始创建自己的开源门户 Netflix Open Source (别名 NetflixOSS，http://netflix.github.io/) 了，他们并不知道这会如何发展；也不知道开源贡献者是否会使用，改进或者是忽略；更没想到的是就这样拥有了公司的社区，开发者会给予反馈，一些中间供应商会把这些开源软件集成到他们的解决方案中。

到了今天，Netflix 拥有上百个开源软件，从基础设施平台到大数据工具，再到自动化部署软件。随着时间的发展，OSS 网站也由于越来越多的组件而变得越来越复杂。现在，Netflix 还会继续开源更多的软件。

Netflix 发布在 Github 中的项目库地址：https://github.com/netflix

三、 Spring Cloud 框架结构



四、 Spring Cloud 和 Dubbo 的对比

Spring Cloud 和 Dubbo 都是微服务开发框架。不是新的技术就一定是好的技术。Dubbo 优势在于开发简单，效率高。Spring Cloud 优势在于功能全面且可靠性高。

对比内容		Dubbo	Spring Cloud
出身		阿里系 核心框架是服务化治理	Spring 社区 核心框架是 Netflix 开源微服务架构群体
文档		集中，健全，中文	较多，内容大部分是英本版
性能		Dubbo 的性能大约是 Spring Cloud 的 2~3 倍	
功能	服务注册中心	zookeeper	Spring Cloud Netflix Eureka
	服务调用方式	RPC	REST API

	服务网关	无	Spring Cloud Netflix Zuul
	断路由	集群容错	Spring Cloud Netflix Hystrix
	分布式配置	无	Spring Cloud Config
	服务跟踪	无, monitor	Spring Cloud Sleuth
	消息总线	无	Spring Cloud Bus
	数据流	无	Spring Cloud Stream
	批量任务	无	Spring Cloud Task

性能对比资料：

https://mp.weixin.qq.com/s?__biz=MzA5MzQ2NTY0OA==&mid=2650796496&idx=1&sn=a544b76660484b9914b65f038cc39e6d&chksm=88562c8fbf21a5995909ffa9f172f31651b1ebd04897917e43caef3491954e24ed0d0477a5a1&mpshare=1&scene=23&srcid=01245faqrBlQETYK9c7zVmd3#rd

五、 Spring Cloud 版本号说明

1 常见版本号说明

开发中，使用的框架版本，最好是 RELEASE 版本或 Final 版本。

常见版本号格式为： x.y.z.stage

x - 数字格式主版本号，当功能模块有较大更新或者整体架构发生变化时，主版本号会更新。

y - 数字格式次版本号，次版本表示只是局部的一些变动。

z - 数字格式修正版本号，一般是 bug 的修复或者是小的变动。

stage - 希腊字母版本号，也称为阶段版本号。用于标注当前版本的软件处于哪个开发阶段。常用的阶段版本包括：BASE、ALPHA、BATE、RELEASE/FINAL。

BASE - 设计阶段。只有相应的设计没有具体的功能实现。

ALPHA - 软件的初级版本。存在较多的 bug。

BATE - 表示相对 ALPHA 有了很大的进步，消除了严重的 bug，还存在一些潜在的 bug。

RELEASE/FINAL - 该版本表示最终版，即正式发布版本。

2 Spring Cloud 版本号说明

Spring Cloud 是一个包含若干子框架的框架集合体，是一个完整的微服务框架体系，如果使用场景版本号来进行标记，容易混淆主框架版本和子框架版本标记。所以 Spring Cloud 使用一种全新的版本号来对主框架进行版本标记，而子框架的版本标记大多还是使用常用版本号标记的。

Spring Cloud 版本格式如： 版本号命名.stage

版本号命名：Spring Cloud 主框架版本号是使用英国伦敦地铁站名称来进行标记的，并根据地铁站名称的首字母的英文自然升序排列来识别版本的递增。如：Angle、Brixton、Camden、Dalston、Edgware、Finchley 等。后续版本提升会继续根据首字母升序排列。

stage：阶段版本号。常用的阶段版本包括：BUILD-XXX[SNAPSHOT]、GA、PRE(M1、

M2 等)、RC、SR。

BUILD-XXX[SNAPSHOT] - 开发版本、一般是开发团队内部使用。

GA - 稳定版，内部开发到一定阶段了，各个模块集成后，经过全面测试发现没有问题，可对外发行了。这个时候叫 GA(General Availability)。基本上可以使用了。没有严重的 BUG 问题，但是有未测出的 BUG 隐患。不推荐商业使用。

PRE - 里程碑版，由于 GA 还不属于公开发行版，里面还有些功能不完善或者 bug，于是就有了 milestone（里程碑版）。milestone 版主要修复了一些 bug。一个 GA 后，一般会有多个里程本版。例如 M1 M2 M3.....。不推荐商业使用。

RC - 候选发布版，从 BUILD 后到 GA 在到 M 基本上系统就算定型了，这个时候系统就进入 Release Candidate（候选发布版）。该阶段的软件类似于最终发行前的一个观察期，该期间只对一些发现的等级高的 bug 进行修复。发布 RC1 RC2 等版本。可以考虑 RC 版本。

SR - 正式发布版，公开正式发布。正式发布版一般也有多个发布，例如 SR1 SR2 SR3 等等，一般是用来修复大 bug 或者优化。最好使用 SR 版本。

学习过程使用的版本是：Greenwich.SR4。当前版本中内部使用的 SpringBoot 是 2.2.x.RELEASE 版本。